Mid-Evaluation: Principles & Practices

CS5013 : Programming with AI

Project : Brahmaputra Hostel System

Akhilesh K.R (CS26E001) & Mysha Ali (CS26E008)

1. Introduction

The Brahmaputra Hostel System aims to simplify hostel room and asset management through structured records, inspections, and asset lifecycle tracking. All generated code is reviewed and tested before integration to ensure that we understand the implementation and can explain our design decisions.

2. Prompt Engineering

We follow a structured approach to prompting AI tools:

- **Context provisioning:** We provide relevant source code, project structure, dependencies, and error messages instead of asking isolated questions.
- **Clear constraints:** We specify the required technology stack, expected behaviour, and architectural requirements.
- **Iterative refinement:** We break larger tasks into smaller steps, reviewing the proposed design before implementing individual components.
- **Targeted debugging:** We provide exact errors and relevant code, asking for the root cause and a focused fix rather than an unrelated rewrite.
- **Verification:** We review generated code and check that it compiles and behaves as expected before integration.

Sample Prompts

Prompt 1 - Entity and Repository Design

> Based on our Spring Boot and PostgreSQL project, help implement a Room entity for the Brahmaputra Hostel System. Rooms can be single, double, or triple sharing. Include appropriate field constraints and explain how the entity maps to the database. Keep the implementation consistent with the existing package structure.

Prompt 2 - Debugging

> I am running ./mvnw spring-boot:run and encountering this error: [error]. Here is my relevant source code and configuration. Identify the root cause, suggest the smallest appropriate fix, and explain how I can verify that the application starts and connects to PostgreSQL correctly.

Prompt 3 - API Implementation

> Help implement a REST API for managing hostel rooms using Spring Boot. Use a controller, service, and repository structure. The API should support retrieving rooms and creating a room. Explain the role of each layer and provide example requests for testing the endpoints.

3. Debugging

Our debugging process involves reproducing the issue, isolating the relevant error, identifying its root cause, and applying a targeted fix.

- **Error analysis:** We examine Maven compiler output and application logs to identify the first relevant error and trace it to the affected class or configuration.
- **Incremental fixes:** We make focused changes rather than rewriting unrelated components, then rerun the relevant build or application command.
- **Database verification:** We inspect PostgreSQL tables and application logs to confirm that entities are mapped correctly and database connections are established.
- **API validation:** As endpoints are implemented, we will test requests and responses using HTTP clients and verify expected status codes and error handling.

4. Version Control (Git)

Git helps us track implementation progress, collaborate, and recover from faulty changes.

- **Logical commits:** We group related changes into descriptive commits, such as Add Room entity and repository.
- **Safe integration:** Before significant refactoring, we inspect the working tree and preserve valid work. We review diffs and test changes before committing them.
- **Tracking decisions:** Important implementation choices and prompts for major changes can be recorded in project documentation, making the development process easier to review and explain.

- **Recovery:** Git history and targeted revert operations help us recover from regressions without unnecessarily discarding unrelated work.

5. Testing, Refactoring and Documentation

We aim to maintain a modular, testable backend while developing the core hostel asset-management workflow.

- **Layered architecture:** Controllers handle HTTP requests, services contain business logic, repositories manage database access, and entities represent persistent data. This separation improves maintainability and makes individual components easier to test.
- **Build verification:** We use Maven to compile the application and will run automated tests as business logic is added. A successful build is treated as a necessary check, not proof that every workflow is correct.
- **Workflow testing:** By the mid-demo, we aim to demonstrate room and asset management, condition recording, and vacancy inspection using synthetic data. Tests will cover condition changes, missing assets, and preservation of asset history.
- **Documentation:** We maintain project documentation covering the architecture, setup instructions, database configuration, and implemented API endpoints. Documentation will be updated as functionality is completed.

6. Conclusion

Our engineering approach emphasizes structured AI prompting, systematic debugging, disciplined Git usage, modular design, and incremental verification. These practices support the development of a maintainable system while ensuring that the core room, asset, and inspection workflows can be demonstrated reliably. We will continue to expand automated testing and documentation as the project progresses.